

Write a program that works out the smallest number of coins needed to give a customer their change.

\$./cash

Change owed: 0.41

4

Forty-one cents is a quarter, a dime, a nickel and a penny. Four coins.

Your program must:

- Ask for the change owed in dollars, as a float
- Re-prompt if the user gives a negative number
- Use a greedy algorithm - take the largest coin that still fits, as many times as it fits, then move down. Quarters, dimes, nickels, pennies
- Print only the number of coins, and nothing else

Your pseudocode

START

REPEAT

Ask the user for change owed in dollars

Get the change as a float

UNTIL change owed ≥ 0

Convert dollars to cents

Set coins to 0

WHILE cents ≥ 25

Subtract 25 from cents

Add 1 to coins

END WHILE

2. Full source of Cash.c

```
#include <stdio.h>

int main(void)
{
    float change;

    // Ask for change and re-prompt if negative
    do
    {
        printf("Change owed: ");
        scanf("%f", &change);
    }
    while (change < 0);

    // Convert dollars to cents
    int cents = (int)(change * 100);

    int coins = 0;

    // Quarters
    coins += cents / 25;
    cents %= 25;

    // Dimes
    coins += cents / 10;
    cents %= 10;

    // Nickels
    coins += cents / 5;
    cents %= 5;

    // Pennies
    coins += cents;

    printf("%d\n", coins);
}
```

3. Floating-point numbers can sometimes be stored as slightly different values than the number we enter. For example, 0.41 might be stored as something like 0.409999, which could cause the coin

calculation to be incorrect. To deal with this, I convert the dollar amount to cents and round it to the nearest whole cent. The line of code that handles this is `int cents = (int)(change * 100 + 0.5);`.